Mobile Developer Intern Assessment

Take-Home Technical Task & Evaluation Rubric

Key Dates

Milestone	Date
Task issued	Saturday, 11 July 2026
Submission deadline	Thursday, 23 July 2026
Shortlisting decision	Saturday, 25 July 2026
Interviews held	Monday, 27 July 2026
Final decision communicated	Evening, Monday 27 July 2026
Internship start date	Saturday, 1 August 2026

1. Overview

This assessment is used to shortlist candidates for a 2-month mobile developer internship position working on the **CalmAnchor** project — a phased mental-health companion mobile app digitising a CPTSD toolkit for patients and practitioners.

All candidates, regardless of claimed experience level (from final-year students to those with 8-10 years of prior industry experience), will be given the **same task** and evaluated against the **same rubric**. Prior experience on a CV is not scored directly — only demonstrated output in this assessment counts.

Agent-assisted development (e.g. Claude Code, Cursor, Copilot, or similar) is **permitted but must be explicitly acknowledged** in the repository documentation. How well a candidate orchestrates and reviews agent-generated work will be separately evaluated (see Section 6).

Candidates have **9 working days** to complete this task, spanning **13 calendar days** in total (11 July - 23 July 2026). A follow-up email will specify expected time commitment guidance; candidates should manage their own schedule around this.

2. The Task: “CalmAnchor Lite”

Candidates must build a fixed-scenario CRUD mobile application that demonstrates relational database mastery and the mobile engineering skills required for this

internship. This is **not** an open/fictional domain choice — the scenario below is mandatory as the baseline.

Scenario: A single doctor manages one working day of appointments for five patients. There is **no login/authentication required** for either the doctor or patients, and no separate views are needed — the app assumes the doctor is the only user.

Core entities and relationships:

- **Doctor** — a single doctor record (no auth needed)
- **Patients** — five patients, each with basic history/notes linked to the doctor
- **Appointment slots** — the working day runs 9:00 AM to 5:00 PM (8 hours), split into 20-minute slots

Core functional requirements:

- Doctor can view the list of patients and their history
- Doctor can view the day's appointment schedule across all slots
- Doctor can move/reschedule an appointment to a different slot
- When changing an appointment, the form must **not display already-booked slots** as available options
- No rebooking-conflict handling, cancellation flows, or multi-day scheduling required — keep this part simple
- Candidates are encouraged to add functionality beyond this baseline, but the above is the mandatory minimum

Technical requirements:

- Cross-platform mobile app (React Native strongly recommended, alternatives allowed if justified)
- Minimum 4-5 screens: Patient List, Patient Detail, Day Schedule/Home, Change Appointment form, Settings/Profile
- **Cloud database preferred**: MongoDB (Atlas), Firebase, or Supabase — cloud options preferred over local SQLite since setup is simpler and, in Supabase's case, auth is available if candidates want to extend scope
- Full CRUD demonstrated across Doctor, Patient, and Appointment entities, with clear relational mapping between them

3. Documentation Requirements

All documentation lives in a /docs folder inside the repository, not as a separate submission. **Diagrams are strongly encouraged over plain text/Markdown** for user journeys and screen maps — candidates should use tools like Lucidchart (or equivalent), export as images, and embed them in the docs. A clear diagram is easier to mark than a wall of text.

- **User journey** — diagrammed (image) description of what the doctor does and why across the working day
- **Screen map** — diagrammed list of screens and how they connect to one another
- **Database schema** — entities, fields, relationships (ERD image preferred; markdown tables acceptable as a supplement)
- **README** — setup, run instructions, and APK build steps
- **Architecture note** — folder structure, state management choice, rationale for database choice
- **Agent usage disclosure** — if AI coding agents were used, briefly document where/how (mandatory only if used)

4. Structured Commit & Branching Plan

Candidates must demonstrate a **clear, sequential, and well-paced progression** of commits across the assessment window — not a small number of large commits clustered near the deadline. Reviewers will check commit *timestamps* for even distribution across the working period.

Required progression (in order, each as one or more distinct commits/branches):

1. Repository setup + initial documentation (user journey, screen map, DB schema diagrams) committed before feature code begins
2. Home/Day Schedule screen wired to the database, displaying seeded/real data (read-only, no forms yet)
3. Full database layer implemented with raw/unstyled screens rendering data across Doctor, Patient, and Appointment entities
4. Change Appointment form wired to the database, correctly excluding already-booked slots
5. All screens assembled and connected with full navigation, end to end

6. APK exported via Android Studio/Gradle, installed on a **separate** Android emulator (BlueStacks, Genymotion, or a second AVD instance), plus final polish

Branching strategy requirements:

- Repository must be **public** for the full duration of review
- Candidates must use feature branches (not commit everything to main), e.g. feature/db-schema, feature/day-schedule, feature/change-appointment-form
- **Every commit on main must represent a fully working, bug-free state of the software** — main should only receive merges when a feature or task phase is genuinely complete and functional, never mid-work or broken code
- Merge history (merge commits or PRs, even if self-reviewed) should be visible and map logically to the milestone progression above
- Reviewers will assess branch naming, merge frequency, and whether main at any point in time reflects a stable, demoable build

5. Deliverable & Video Requirements

Requirement	Detail
Repository	Single **public** GitHub repo with full, evenly-paced commit and branch history
Cross-platform build	React Native (or justified equivalent) with confirmed iOS + Android export capability
APK export	Built via Android Studio/Gradle, not just run in a dev/Metro emulator
Second emulator install	APK manually installed on a distinct Android emulator (BlueStacks, Genymotion, or separate AVD)
Video	7-10 minute screen recording (Zoom, camera + audio on), showing the app running **only** on the second emulator, plus a walkthrough of the repo documentation
Signing (stretch/bonus)	Bonus points for demonstrating a signed release build process (real Play Store path)

6. Evaluation Rubric (100 points)

Criterion	Weight	Subtasks Assessed	What Costs Marks
Commit & branch quality	25%	Even commit pacing across the window; meaningful commit messages; feature branches used correctly; main only receives complete, working merges	Commits clustered near deadline; vague messages (e.g. “fixes”); broken/incomplete code merged

Criterion	Weight	Subtasks Assessed	What Costs Marks
			to main; everything committed directly to main
Documentation quality	20%	Diagrammed user journey; diagrammed screen map; ERD/schema diagram; clear README with run/build steps; architecture rationale	Text-only docs where diagrams were expected; missing setup instructions; schema doesn't match actual implemented relationships
Functional correctness	25%	Full CRUD across Doctor/Patient/Appointment; correct exclusion of booked slots when rescheduling; APK installs and runs without crashing on a separate emulator	Booked-slot filtering logic missing or wrong; app crashes on second emulator; CRUD operations incomplete or broken
Code quality & architecture	15%	Sensible folder structure; component reuse; appropriate state management; clean handling of relational data between entities	Unstructured/dumped code; obvious unreviewed AI boilerplate; no separation of concerns
Communication in video	10%	Clear explanation of design decisions; confident walkthrough of app and docs within 7-10 minutes	Video over/under time limit; candidate can't explain their own code; unclear audio/camera not on
AI agent orchestration (bonus)	+5%	Quality of prompting/orchestration if agents used; evidence of human review/correction of agent output; disclosure completed	Agent use undisclosed; no evidence of review (raw unedited agent output); disclosure present but orchestration shows no real understanding

Scoring is deliberately **blind to CV-stated seniority** — a candidate with 10 years of experience and a candidate with none are judged purely on this output.

7. Hiring Timeline

See Key Dates table at the top of this document.